QuantSolve, explained

What the project does, how one run works, and what we measured

Deon Menezes (16014223030) · **Aishwarya Gawali** (16014223007) · Batch B1 · Generative AI Laboratory, Lab CA
Code: https://github.com/DeonMenezes-11/QuantSolve

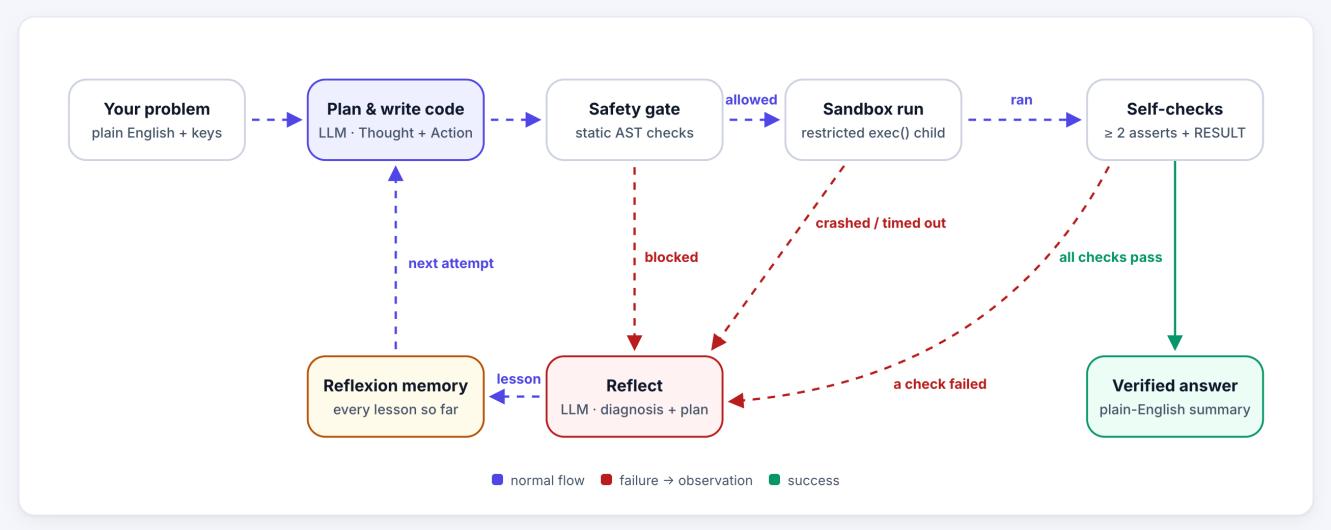
1. What QuantSolve is

QuantSolve answers multi-step **numbers questions** (loans, investments, income tax, physics, probability, inventory, counting). Instead of asking a language model for the number directly, it asks the model to **write a Python program** that computes the answer. The program runs in a locked-down **sandbox**, it has to **check its own answer** with at least two `assert` statements, and when anything goes wrong the model **reads the error, writes down a lesson and tries again**, up to five attempts. The answer you get back is therefore one that came out of code which actually ran and passed its own checks.

2. Why we built it this way

Language models write code that looks right but often contains small slips: a loop that stops one step early, a flipped sign, a rule from the question applied in the wrong order. A single attempt hides those slips. Running the code gives an objective signal (the interpreter does not care how confident the model sounded), and feeding that signal back lets the model repair its own work. This is the **Self-Correction "Code Execution & Auto-Heal" loop** from the lab brief, built on two published ideas: **ReAct** (reason, act, observe) and **Reflexion** (learn from failures through written self-reflection).

3. The loop in one picture



Blue arrows are the normal flow and red arrows are failures, which become the observation for the next attempt. A lesson is stored after every failure and shown to every later attempt. Green means every self-check passed.

4. One real run, step by step

Run `20261006-024806_income_tax_inject-crash`: income tax under India's new regime (FY 2024-25) for three salaries, with the *crash* demo switch on, so QuantSolve deliberately breaks the first program to show the repair.

1. **Thought + Action.** The model plans the method (taxable income, slab rates, rebate under 87A, marginal relief, 4% cess) and writes a 39-line program that stores the answer in `RESULT` and contains 5 self-checks.
2. **Demo bug.** QuantSolve renames one use of `standard_deduction` to `standard_deductino` on line 3, exactly as a typo would.
3. **Observation.** The sandbox reports a **NameError** on line 3, with a traceback trimmed to the program's own lines.

4. **Reflection.** The model writes: *"You must carefully check variable names for typos, especially when they are defined and used closely together, to avoid runtime errors that halt execution."* This lesson is kept in memory.
5. **Attempt 2.** The program is rewritten with the fix. All 5 self-checks pass and the answer matches the hand-written reference: tax A = Rs 0, tax B = Rs 15,600, tax C = Rs 2,31,400, effective rate 12.51%. The whole run took 15 s and cost about $0.0051.

5. The ideas behind it

Idea	What it means	Where it lives in QuantSolve
ReAct	Reason (Thought), act (Action), look at the result (Observation), repeat.	Each attempt = a plan, a full program, and the sandbox result.
Reflexion	After a failure, write a short lesson and remember it.	A separate call returns diagnosis, lesson and next plan; all lessons go into every later prompt.
Self-verification	The program checks its own answer.	At least two asserts comparing quantities computed two ways (loop vs formula, parts vs total).
Program-aided reasoning	Let Python do the arithmetic.	The model never states the final number; the program computes it.
Sandboxing	Run untrusted code without risk.	AST safety gate, restricted builtins, guarded import, separate process with no API key, CPU and memory limits, timeout.

The sandbox was attacked with 17 escape and resource-exhaustion programs (shell commands, reading system files, introspection tricks, infinite loops, memory bombs, reading the API key). **All 17 were contained.** Three of them ran with the static gate switched off on purpose, to show that the runtime layers hold on their own.

6. The website

Started with `python3 web.py`; it runs on your own machine at `http://127.0.0.1:8503`. It is a hand-built single-page site (FastAPI on the server, plain HTML, CSS and JavaScript in the browser) with light and dark themes, and it works on a phone.

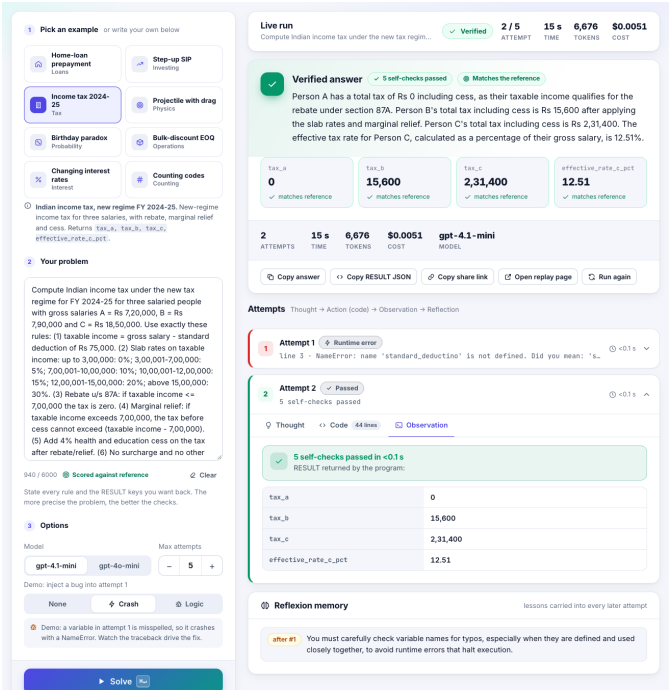
Solve: pick one of eight examples or type a problem, choose the model, the attempt budget and an optional demo bug, then watch every attempt stream in live with its Thought, Code (failing line highlighted), Observation and Reflection.

History: every saved run, searchable; coloured squares show each run's path (crash, then fixed, and so on).

Run replay: any saved run, with a shareable link.

Results: the benchmark numbers, charts, model comparison and safety table.

How it works: the diagram above, the ideas and the sandbox layers.



Right: the Solve page after the run described in Section 4.

7. What we measured

Eight problems with hand-written reference answers, each run 2 times per setting, at most 5 attempts. A run counts as correct only if every value matches the reference within tolerance.

Setting	Correct answers (gpt-4.1-mini)	What it shows
One attempt, no healing	69% (11/16)	The baseline.
The full loop	81% (13/16)	Healing raises accuracy by 12 points, using 1.19 attempts on average.
Loop + injected crash	83% (5/6)	Crashes are easy to repair: 5 of 6 runs recovered.
Loop + injected logic bug	50% (8/16)	Harder: the self-checks caught 2 of 9 logic bugs that could be traced; the rest slipped through or changed nothing.
Weaker model (gpt-4o-mini)	6% → 31%	The loop helps, but how much depends on how well the model can debug itself.

A loop run cost about **$0.0029** on average. The honest limitation: checks written by the same model can share its blind spots, so a program can pass its own checks and still be wrong. That is why every sample problem is also scored against an independent reference answer.

8. Try it

`pip install -r requirements.txt` → put an OpenAI key in `.env` → `python3 web.py` for the website, or `python3 cli.py run-task income_tax --inject crash` in a terminal. `python3 -m pytest` runs the offline test suite. Full code, results and the reports: https://github.com/DeonMenezes-11/QuantSolve